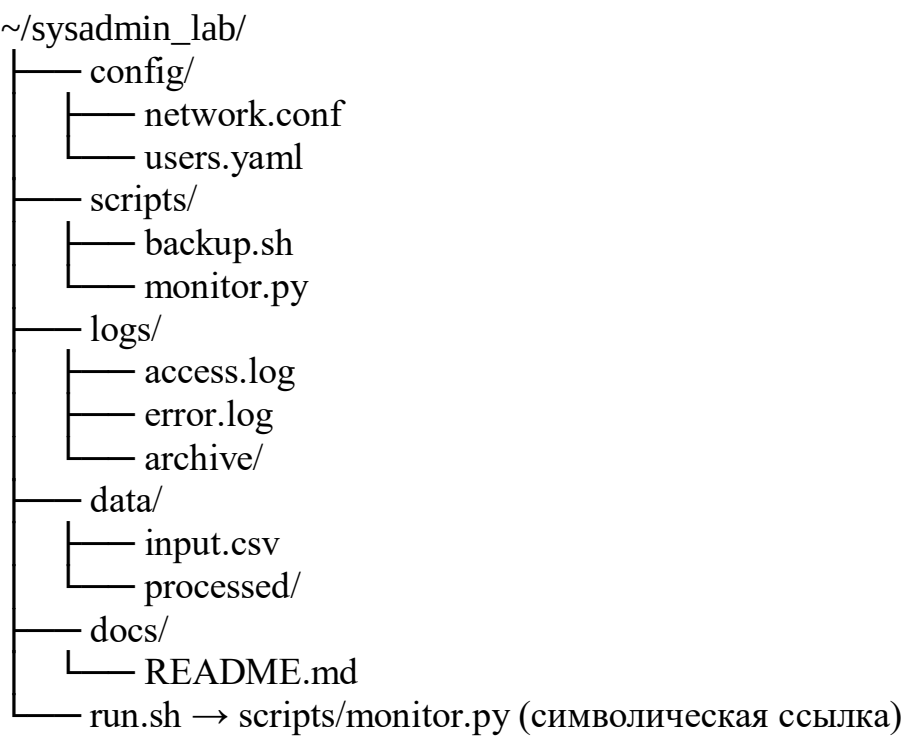


ЗАДАНИЕ: Создание иерархической структуры каталогов и файлов с использованием команд терминала

Цель задания:

Отработать навыки создания сложных файловых структур, генерации файлов заданного типа и размера, работы с символическими ссылками и проверки корректности созданной иерархии в РЕД ОС.

В домашнем каталоге пользователя необходимо воссоздать следующую структуру проекта:



ТРЕБОВАНИЯ К ФАЙЛАМ И КАТАЛОГАМ

Объект	Требование
backup.sh	Содержит shebang <code>#!/bin/bash</code> и комментарий <code># Скрипт резервного копирования</code>
monitor.py	Содержит shebang <code>#!/usr/bin/env python3</code> и комментарий <code># Мониторинг системы</code>
network.conf	Текстовый файл минимум из 4 строк конфигурации сети
access.log, error.log	Размер каждого ≈ 10 КБ (заполнить текстовыми строками)
run.sh	Относительная символическая ссылка на <code>scripts/monitor.py</code>

Объект	Требование
.sh файлы	Права на выполнение для владельца (rwx----- или rwxr-xr-x)

Все команды выполнять в терминале. Перед началом убедитесь, что находитесь в домашнем каталоге: `cd ~`

Создание структуры каталогов

```
mkdir -p ~/sysadmin_lab/{config,scripts,logs/archive,data/processed,docs}
```

-p создаёт родительские каталоги при необходимости, { . . . } позволяет задать несколько путей одной командой.

Создание пустых файлов

```
touch
~/sysadmin_lab/{ config/network.conf,config/users.yaml,scripts/backup.sh,scripts/
monitor.py,logs/access.log,logs/error.log,data/input.csv,docs/README.md}
```

Заполнение файлов содержимым

```
# Заголовки скриптов
echo -e '#!/bin/bash\n# Скрипт резервного копирования' >
~/sysadmin_lab/scripts/backup.sh
echo -e '#!/usr/bin/env python3\n# Мониторинг системы' >
~/sysadmin_lab/scripts/monitor.py
```

```
# Конфигурация сети
cat << EOF > ~/sysadmin_lab/config/network.conf
[interface]
name = eth0
ip = 192.168.1.10
mask = 255.255.255.0
EOF
```

Генерация лог-файлов заданного размера

```
# Заполняем access.log ~10 КБ текстовыми строками
yes "$(date +"%Y-%m-%d %H:%M:%S") INFO: Connection established" | head -
c 10K > ~/sysadmin_lab/logs/access.log
```

```
# Аналогично для error.log
yes "$(date +"%Y-%m-%d %H:%M:%S") ERROR: Timeout on port 8080" | head
-c 10K > ~/sysadmin_lab/logs/error.log
```

Создание символической ссылки

```
ln -s scripts/monitor.py ~/sysadmin_lab/run.sh
```

Относительная ссылка предпочтительнее: она останется рабочей при перемещении всей папки проекта.

Настройка прав доступа

```
chmod u+x ~/sysadmin_lab/scripts/backup.sh ~/sysadmin_lab/run.sh
```

ПРОВЕРКА РЕЗУЛЬТАТА

Визуализация структуры (установите **tree**, если отсутствует: **sudo dnf install tree**): **tree ~/sysadmin_lab**

Подробный список с указанием ссылок и прав: **ls -lR ~/sysadmin_lab**

Проверка типа файла и содержимого shebang:

```
file ~/sysadmin_lab/scripts/backup.sh
head -n 1 ~/sysadmin_lab/scripts/monitor.py
```

Проверка размера лог-файлов: **ls -lh ~/sysadmin_lab/logs/*.log**

Проверка работоспособности ссылки: **readlink -f ~/sysadmin_lab/run.sh**
cat ~/sysadmin_lab/run.sh

КОНТРОЛЬНЫЕ ВОПРОСЫ К ЗАДАНИЮ

1. Почему использование **mkdir -p** и **brace expansion {...}** считается лучшей практикой при создании сложных структур?
2. В чём разница между **ln** (жёсткая ссылка) и **ln -s** (символическая ссылка)? Когда какую следует применять?
3. Почему команда **yes ... | head -c 10K** предпочтительнее **dd if=/dev/urandom** для создания текстовых логов?
4. Как команда **chmod u+x** влияет на биты прав в восьмеричной системе? Запишите результат в формате 755.
5. Что произойдёт, если удалить оригинальный файл **scripts/monitor.py**? Станет ли ссылка **run.sh** рабочей?

Задание 1. Просмотр информации о дисках и разделах

1. Откройте терминал.
2. Выведите список всех блочных устройств: `lsblk`
3. Получите подробную информацию о файловых системах: `df -h`
4. Выведите UUID разделов: `blkid`
5. Зафиксируйте в отчёте вывод команд и поясните значения столбцов NAME, SIZE, TYPE, MOUNTPOINT (для `lsblk`) и Filesystem, Size, Used, Use%, Mounted on (для `df`).

Задание 2. Создание тестового файла-диска и раздела

(Безопасный метод, не затрагивает реальные носители)

1. Создайте файл размером 500 МБ: `dd if=/dev/zero of=~/.test_disk.img bs=1M count=500`
2. Свяжите файл с устройством `loop`:
`sudo losetup -f ~/.test_disk.img`
`losetup -a` # запомните имя устройства, например `/dev/loop0`
3. Создайте раздел на `loop`-устройстве с помощью `fdisk`: `sudo fdisk /dev/loop0`
 - Введите `n` (новый раздел), `p` (основной), `1` (номер)
 - Оставьте начальный и конечный сектор по умолчанию
 - Введите `w` для сохранения и выхода
4. Перечитайте таблицу разделов (если требуется): `sudo partprobe /dev/loop0`
(Если `partprobe` отсутствует, установите: `sudo dnf install parted`)

Задание 3. Создание файловой системы и монтирование

1. Отформатируйте созданный раздел в `ext4`: `sudo mkfs.ext4 /dev/loop0p1`
2. Создайте точку монтирования: `sudo mkdir -p /mnt/lab_fs`
3. Смонтируйте раздел: `sudo mount /dev/loop0p1 /mnt/lab_fs`
4. Проверьте результат:
`mount | grep lab_fs`
`df -h /mnt/lab_fs`
`ls -la /mnt/lab_fs`

Задание 5. Управление правами доступа

1. Создайте файл от имени текущего пользователя:

`touch /mnt/lab_fs/docs/readme.txt`

`echo "Тестовый файл" > /mnt/lab_fs/docs/readme.txt`
2. Просмотрите текущие права:

`ls -l /mnt/lab_fs/docs/readme.txt`

3. Измените права: запретить запись для группы и остальных, разрешить выполнение владельцу:

```
chmod 754 /mnt/lab_fs/docs/readme.txt
```

```
ls -l /mnt/lab_fs/docs/readme.txt
```

4. Смените владельца файла на root:

```
sudo chown root:root /mnt/lab_fs/docs/readme.txt
```

```
ls -l /mnt/lab_fs/docs/readme.txt
```

5. Попробуйте отредактировать файл от имени обычного пользователя. Зафиксируйте результат.